Computational Neuroscience — Week 2

Neural Code and Neural Encoding

Reading the Code, Modelling It, and Predicting Spikes

These notes cover lectures 1–3. Where the lecture states a result quickly, I have tried to unpack why *it is true — green boxes give the plain-words version of each idea before the mathematics, and rust boxes flag the places where the obvious reading is the wrong one.*

Contents

I What Is the Neural Code? 2

1 The Question 2

2 Recording from the Brain 3

3 A First Look at the Code: the Retina 4

4 Encoding and Decoding 6

5 Choosing a Stimulus Parameter 6

6 Tuning Curves 7

7 Functional Maps 8

8 Higher-Order Feature Selectivity 8

9 A Hierarchy of Complexity — and Its Limits 9

II Building a Response Model 10

10 Setting Up the Problem 10

11 The Simplest Possible Model: Linear Response 10

12 Temporal Filtering 11

13 Spatial Filtering 11

14 Spatiotemporal Filtering 14

15 Why a Linear Filter Cannot Be the Whole Story 14

16 The Linear–Nonlinear Model 15

III Finding the Model from Data 15

17 The Problem Is Dimensionality 16

18 Why White Noise? 17

19 The Spike-Triggered Average 17

20 Recovering the Nonlinearity 18

21 Multiple Features: Principal Component Analysis 19

22 Maximally Informative Dimensions 20

IV From Firing Rates to Spikes 22

23 The Assumption We Have Been Hiding 22

24 From Coin Flips to the Poisson Distribution 22

25 Testing It Against Real Data 24

26 Generalized Linear Models 25

27 The Time-Rescaling Theorem 27

28 What These Models Leave Out 27

Part I

What Is the Neural Code?

Lecture 1 — recording techniques, the retina, encoding vs decoding, tuning curves, and where a tuning curve runs out.

1. The Question

Brains do many things, but one of the best studied is the **representation and transformation of sensory information**. Responding to the world — or just thinking about it with your eyes shut — requires converting information from one physical form into another. Light, sound, or written language becomes some process inside the head that carries a version of that information.

A natural way to talk about this is as a **code**. The task is to discover the form in which information is represented in neural activity.

The neural code

The mapping between stimuli in the world (or internal variables, such as an intended movement) and the patterns of neural activity they produce.

The idea in plain words

A neuron's only output is spikes, and every spike looks like every other one. So whatever a neuron is "saying," it cannot be in the shape of an individual spike — it has to be in

which spikes happen and *when*. Cracking the code means working out the rule that turns a stimulus into that pattern.

The method used to record brain activity determines what analysis is possible and what information that activity can contain. So the sensible starting point is not theory but instrumentation.

2. Recording from the Brain

Running from large scale to small scale — and, roughly, from *non-invasive but blurry* to *invasive but precise*. That trade-off is the single most important thing to take from this section.

fMRI — functional magnetic resonance imaging

[Lecture slide omitted from the public copy.] An fMRI scanner. The subject's head lies at the centre of the magnet. Resolution is millimetres and seconds, so every measurement pools the activity of hundreds of thousands of neurons. Scott Murray.

The subject lies with their head at the centre of a large magnet. The scanner measures spatial perturbations in the magnetic field caused by changes in **blood oxygenation**: as a brain region becomes active, blood flows in to support the underlying neural activity.

- **Spatial resolution:** about a cubic millimetre — necessarily the average activity of *millions* of neurons.
- **Temporal resolution:** slow. The blood-flow response unfolds over *seconds*; spikes last milliseconds.
- **Constraint:** the task must not involve moving around.

Watch out — fMRI does not measure neural activity

It measures a slow *vascular consequence* of neural activity. Blood arriving in a region is evidence that neurons there were busy a second or two ago — it is a downstream proxy, not the signal itself.

EEG — electroencephalography

[Lecture slide omitted from the public copy.] EEG. Electrodes on the scalp pick up voltage fluctuations generated by large populations of synchronously active neurons. Millisecond timing, but the skull blurs the spatial picture badly.

Electrodes on a cap make contact with the scalp. EEG still averages over many neurons, but it is much *faster* than fMRI, because it captures changes in the electrical fields of the underlying circuits **directly** rather than through blood flow.

The cost is noise. Many sources contribute to the recorded signal, and it must pass through skull and scalp before reaching the electrode.

Multi-electrode arrays

[Lecture slide omitted from the public copy.] A multi-electrode array (Alan Litke, UCSC). Left: the recording hardware. Right: the electrode grid, and a piece of retina laid over it. Many neurons at once, each with single-spike resolution.

Ideally we want single neurons. Where there is direct access to tissue, a multi-electrode array can be used — the classic example built by the physicist Alan Litke. Most of the visible device is amplification electronics; the array itself is tiny:

- each electrode about **10 μm** across — roughly the size of one neuron;
- **512** electrodes, spaced **60 μm** apart;
- many neurons recorded *simultaneously*.

This works when tissue can be laid directly on the array surface — brain slices, or an isolated retina. To record during normal behaviour you instead *penetrate* the brain with a prong-style array (the Utah array; the whole device is about 2 mm long). Each prong tip is an active electrode. Newer designs let prongs be moved individually to hunt for active cells, and some carry several contacts along their length so multiple depths are recorded at once.

Calcium imaging

[Lecture slide omitted from the public copy.] Calcium imaging (Alex Kwan). Left: a field of cells, each one outlined. Right: one fluorescence trace per cell. Every spike lets calcium in, so brightness tracks activity — slowly, but for hundreds of identified cells simultaneously.

Cells are loaded with a **calcium indicator** whose fluorescence changes when calcium binds to it. Calcium enters the cell during action potentials, so fluorescence intensity becomes a record of firing. This can capture very many cells at once — potentially thousands — and fibre optics allow imaging deep inside the brain.

Patch clamp — looking *inside* the cell

[Lecture slide omitted from the public copy.] Patch clamp. A glass micropipette seals onto the membrane, giving direct access to the voltage inside a single cell and to currents through individual channels. The most precise method, and the most invasive. Theresa Knott, Creative Commons.

Every method above measures the electric field *outside* cells. A patch electrode is clamped onto the membrane, making direct electrical contact with the cell interior. This is how we learn how the signals are generated in the first place, rather than merely that they occurred.

> **The trade-off in one line.** Non-invasive methods work on healthy humans but average over millions of neurons and are slow or noisy. Invasive methods resolve single cells and single spikes, but need access to the tissue.

3. A First Look at the Code: the Retina

Your eyes are an outlying but vital part of your brain. The **retina** is a sheet of cells at the back of the eyeball that converts light focused through the lens into electrical signals. Its output is a good first look at the language of the brain.

The experiment

[Lecture slide omitted from the public copy.] The retina preparation. A patch of retina is placed on a multi-electrode array and a film is projected onto it, so the stimulus is known exactly while ganglion cell spikes are recorded. Left: Cajal's 1901 drawing of the retina.

A retina is dissected from an eye and laid on a multi-electrode array, in fluid that keeps it alive. A **movie is projected down onto the retina**, and its neurons respond.

Ramón y Cajal's 1901 drawings — very early connectomics — show the layered circuit: **photoreceptors** (rods and cones) capture light, successive layers process their signals, and the **retinal ganglion cells** form the output layer, their axons joining the optic tract out of the eye.

Raster plots

[Lecture slide omitted from the public copy.] A raster plot (Michael J. Berry II). Each row is one repeat of the same film; each dot is a spike. Reading down a column shows how reliably the cell fires at that moment of the stimulus.

Raster plot

Each tiny dot is one action potential. Time runs horizontally. Each horizontal strip is **one repetition** of the same stimulus, stacked upward. So the vertical axis is *trial number*, not neuron identity.

Play the movie once and the cell fires at particular times. Repeat it, plot the second repetition above the first, and repeat many times over. What emerges is that each cell fires at *specific, reproducible points in the movie*. Some responses are strong, some weak, and on some repeats the cell does not fire at all.

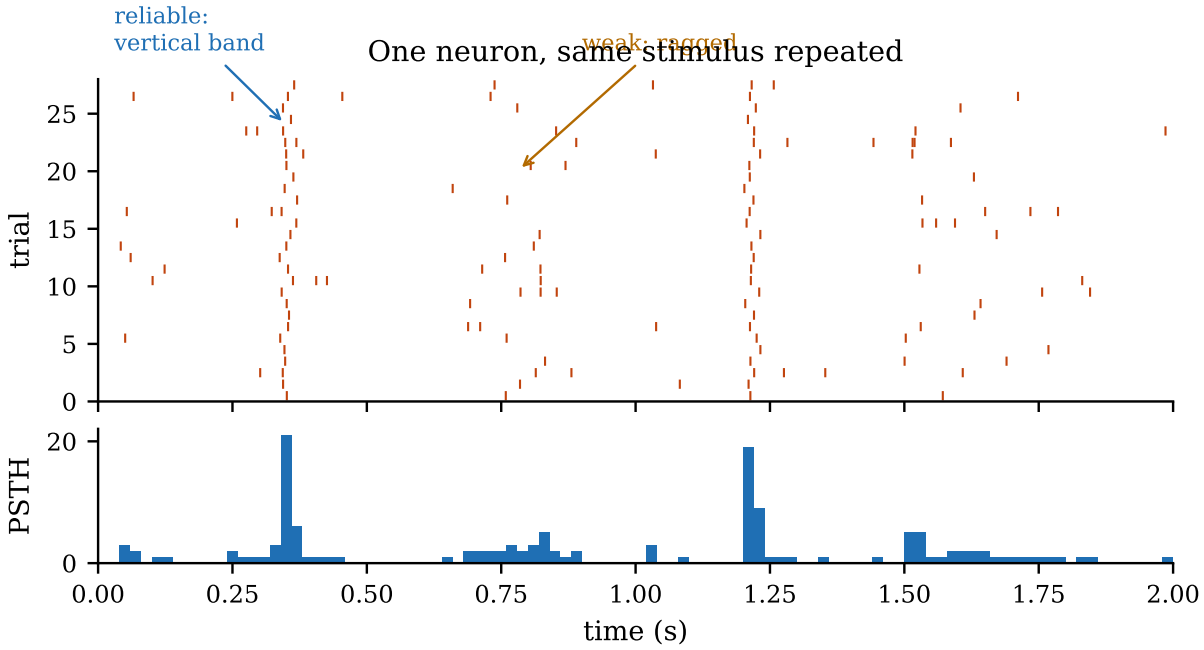


Figure 1: ***Raster plot and PSTH.*** *Each row is one repetition of the same stimulus and each tick is a spike, so the vertical axis is* trial*, not neuron. Where the cell responds reliably the ticks line up into a vertical band; a weaker response gives a ragged, partly-filled band. Summing down the columns gives the PSTH below.*

Recording ∼20 ganglion cells at once shows something genuinely exciting: each neuron encodes some feature or features of the movie, and different cells encode different — sometimes overlapping — sets. Two cells may share many features and still differ sharply on others.

Watch out — the vertical axis is trials, not cells

This is why "many neurons fire together at that time" is wrong. Within one raster plot you are looking at **one** neuron, many times over. A vertical bar is one cell being reliable, not many cells agreeing.

This raises a question the course will not fully settle: does every neuron signal its own message, or does the population respond as a whole in some joint code?

4. Encoding and Decoding

There are two related ways to read the mapping, and they lead in different directions.

Encoding — *the forward problem*

How does a stimulus cause a pattern of responses?

$$P(\text{response} \mid \text{stimulus})$$

This leads toward building **quasi-mechanistic models** of the neural system that let us predict its response.

Decoding — *the inverse problem*

What do the responses we observe tell us about what the stimulus was?

$$P(\text{stimulus} \mid \text{response})$$

Here we can stay agnostic about mechanism. This is the approach behind a **neural prosthetic** — reading neural data and moving a robot arm in the person's intended direction.

The idea in plain words

Read the bar as "given." $P(\text{response} \mid \text{stimulus})$ is *"if I show this, what spikes do I get?"* — the experimenter's question. $P(\text{stimulus} \mid \text{response})$ is *"I saw these spikes; what was out there?"* — which is the situation every downstream neuron is actually in, since spikes are all it ever receives.

Both are *conditional probability distributions*, and deliberately so. Neural systems are noisy and may carry information about things the experimenter does not control, so the model is inherently probabilistic rather than a deterministic function.

Three questions that organise everything that follows.

1. What is the appropriate measure of **response**?
2. What is the right way of thinking about the **stimulus**?
3. What is the **relationship** between them that our coding model embodies?

5. Choosing a Stimulus Parameter

To make progress we look for a **stimulus parameter** along which the response varies. Throughout, this is called s — and its meaning changes from example to example.

For the response, the simplest choice, and the one taken here, is an **average firing rate** (equivalently, a probability of generating a spike).

Picking s well is a real part of experimental design: choose a natural set of parameters and hold everything else as constant as possible. In lower cortical areas this is often easy — the location of a spot of light for an LGN neuron. In higher areas it can be very hard to know what parameter a neuron is even tuned to.

6. Tuning Curves

Tuning curve

A plot of firing rate f against the stimulus parameter s. The value of s giving the maximum is the neuron's **preferred stimulus**.

Gaussian — a V1 neuron

[Lecture slide omitted from the public copy.] A V1 neuron's Gaussian tuning curve. Left: responses to bars at different orientations. Right: firing rate against orientation angle, peaked at the preferred value and falling away on both sides. From Dayan and Abbott.

V1 neurons respond to oriented bars of light passing through a particular location in the visual field. As the bar rotates from almost horizontal to almost vertical, the neuron at first does not respond, then fires faster and faster, then falls away again past its preferred orientation.

Counting spikes in a bin of, say, 100 ms and plotting count against orientation gives a **Gaussian**-looking curve:

$$f(s) \;=\; r_{\max}\,\exp\!\left(-\tfrac{1}{2}\,\frac{(s-s_{\text{pref}})^2}{\sigma^2}\right)$$

Cosine — a motor cortex neuron

[Lecture slide omitted from the public copy.] A motor cortex neuron's cosine tuning curve. Rasters for eight reach directions, and firing rate against movement direction. Broad, and never silent — the cell fires somewhat for every direction. Adapted from Georgopoulos et al. '92.

A monkey makes arm movements at various angles relative to its body. A motor cortical neuron fires fast for some directions and slowly for others, and firing rate against movement direction traces a **cosine**:

$$f(s) \;=\; r_0 + r_{\max}\cos(s - s_{\text{pref}})$$

The idea in plain words

Why different shapes? Orientation is defined over a limited range and a V1 cell cares about a narrow band of it — hence a bump. Movement direction wraps all the way round a circle, and a motor neuron is broadly tuned, active for half the directions and suppressed for the other half — hence a full cosine cycle rather than a narrow peak.

Both examples come from Dayan and Abbott's *Theoretical Neuroscience* — the V1 data originally from Henry et al. (1974), the motor cortex data from Georgopoulos et al. (1992).

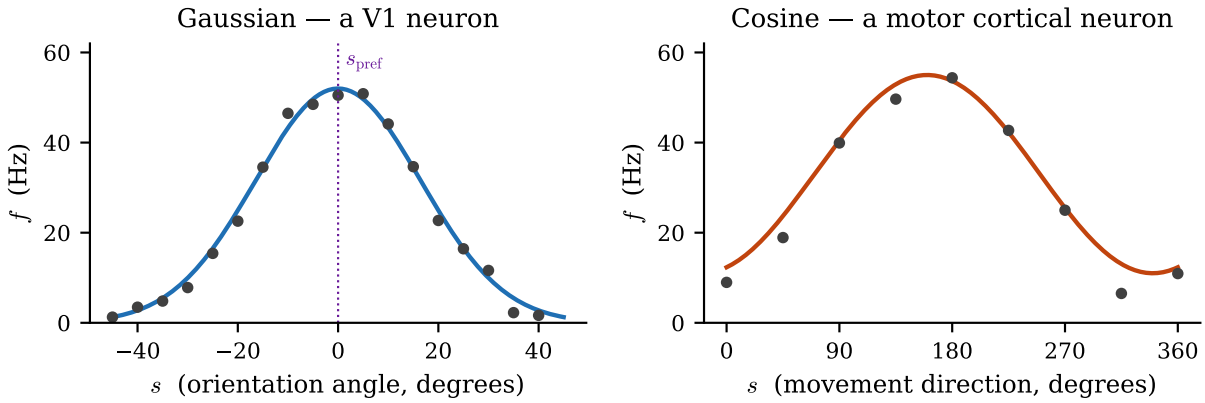


Figure 2: ***Two tuning curves.*** *Left: a V1 neuron's firing rate against bar orientation traces a Gaussian bump around its preferred orientation. Right: a motor cortical neuron's rate against reach direction traces a full cosine, because direction wraps around a circle and the cell is broadly tuned. Points are simulated measurements; curves are the fitted forms in the text.*

7. Functional Maps

[Lecture slide omitted from the public copy.] Functional maps in primary visual cortex (Issa et al. 2008), cat and bush baby. Colour codes preferred orientation (top) and preferred spatial frequency (bottom). Selectivity is laid out smoothly across the cortical sheet rather than scattered at random.

V1 neurons are sensitive to many input features, and their sensitivities are often laid out in an **orderly way across the cortical sheet**.

Functional map

An image of a patch of cortex in which colour encodes the feature that neurons at each location respond to most strongly — response as a function of *position in cortex*.

Maps of preferred orientation show the classic **pinwheel structures**, in which preferred angle rotates systematically across space. Maps of preferred **spatial frequency** — the width of lines in a grating — show a similarly orderly pattern. Both appear in cat and in bush baby (Issa et al., *J. Neurophysiol.*, 2008), which suggests the organisation is not an accident of one species.

8. Higher-Order Feature Selectivity

[Lecture slide omitted from the public copy.] Higher-order selectivity in humans (Haxby et al., Science 2001). Different regions of ventral temporal cortex respond preferentially to faces and to houses.

fMRI shows localised temporal-lobe regions responding to *semantic categories* — faces, or houses (Haxby et al., *Science*, 2001). So "the stimulus that drives a neuron maximally" can get very complex, and drawing a tuning curve against a meaningful parameter becomes difficult.

Concept cells

[Lecture slide omitted from the public copy.] A tuning curve for a concept. Spike counts from one human medial temporal lobe neuron for each of about ninety pictures: it responds to a few (red) and ignores the rest. Quian Quiroga, Reddy, Kreiman, Koch and Fried, Nature 2005.

Quian Quiroga, Reddy, Kreiman, Koch and Fried (*Nature*, 2005) recorded *single neurons in humans* — normally impossible, but these were patients undergoing surgery for epilepsy who agreed to take part.

Plotting one neuron's response against image number (in no particular order) shows a few images that drive it strongly. Looking at *which* images those were reveals a common property.

A second neuron responded to Pamela Anderson — not only to photographs, but to *drawings* of her, and to her *name written as text*. Later work showed such neurons also respond to audio clips of the person's name.

Why these are called concept cells

The response is invariant to sensory format entirely — photograph, drawing, written name, spoken name. What is represented is not an image but something closer to a **concept**.

9. A Hierarchy of Complexity — and Its Limits

[Lecture slide omitted from the public copy.] The tempting story: centre–surround fields, then oriented bars, then more complex shapes, then houses and landmarks. Labelled by hand during the lecture with the anatomy it is supposed to map onto (retina/LGN, V1, V4) — and with the feedback arrows running backwards along the chain, which is where the straightforward account starts to break down.

Putting the examples in order suggests regions of increasing representational complexity, running from geometric to semantic:

Region	Selective for
Retina, LGN (thalamus)	Very simple receptive fields
V1	Oriented edges
V4	Conjunctions of edges forming contours
Higher areas	Semantic categories — houses, and even specific houses such as the White House

It is tempting to read this as features being agglomerated into progressively more specific ones. Along the way comes increasing **invariance**: higher areas are *less* sensitive to details such as colour or position in the visual field. This idea of hierarchical features assembled in a **feedforward** way underpins many powerful models, including ones enjoying great success in machine vision.

Why the real computation is more interesting

[Lecture slide omitted from the public copy.] Top-down effects. The image is the same arrangement of black blobs before and after you find the Dalmatian, but the neural response to it is not. What a neuron does with an input depends on more than the input.

These regions are **massively interconnected**. The thalamus is usually described as a relay station, taking information from sensory receptors and distributing it onward — but in fact it receives an enormous amount of **feedback** from the very areas it feeds.

So representations feed back to control what information comes through in the first place. Semantics — the meaning or context of an image — can influence its *initial* representation in V1 or V4. There is a role here for learning and expectation: **what you think you are looking at can shape what you actually see**.

Top-down effects

The classic demonstration is an image that at first looks like meaningless black-and-white blobs. Once you have seen what it depicts, you can never again fail to see it instantly. Nothing about the image changed — only your prior.

The lecture flags top-down effects as fascinating and open, then sets them aside: the course sticks mainly to lower-level, geometric representations.

Closing Part I. Tuning curves are the simplest encoding model, and they run out of road exactly where stimuli stop being one-dimensional and static. What we need is a model that takes a stimulus varying in *time and space* and predicts a response. That is Part II.

Part II

Building a Response Model

Lecture 2 — linear filtering in time and space, convolution, receptive fields as filters, and why a filter alone is not enough.

10. Setting Up the Problem

We now take $P(\text{response} \mid \text{stimulus})$ seriously as something to *build*: predict $r(t)$ given a stimulus s.

Two simplifications make this tractable, both relaxed later:

- the **response** means a single spike from one chosen neuron;
- we work with the **firing rate** $r(t)$ — the probability of a spike at a given time — and postpone the model of how spike probability depends on that rate. (Part IV takes it up.)

11. The Simplest Possible Model: Linear Response

What is the simplest relationship imaginable? That the response at each time depends *linearly* on the stimulus at that same time — or at a small fixed delay:

$$r(t) \;=\; \phi\, s(t) \qquad \text{or} \qquad r(t) \;=\; \phi\, s(t-\tau).$$

If this held, $r(t)$ would look just like $s(t)$: the same waveform, scaled by a factor, perhaps shifted a little.

Watch out — a neuron has memory

Real responses depend not on the stimulus at *one* instant but on some **combination of recent inputs**. A cell integrates over a window of the recent past — so a single scaling factor cannot be right.

12. Temporal Filtering

If the response depends on a weighted sum of previous inputs, that is still a linear system — just a more general one. Start at time t, step backwards over past time points indexed by k, and weight the stimulus at each by a factor that *depends on how far back you are looking*:

Linear filter — discrete and continuous

$$r(t) = \sum_{k=0}^{n} s_{t-k}\, f_k \qquad\qquad r(t) = \int_{-\infty}^{t} d\tau\; s(t-\tau)\, f(\tau)$$

The weights f_k — or the function $f(\tau)$ — **are** the linear filter. In the integral, τ is a dummy variable playing the role of k.

The idea in plain words

Read the sum right to left. f_k is a list of importance weights: "the stimulus one step ago matters this much, two steps ago this much, three steps ago this much." To get the response now, look back over the recent stimulus, multiply each past value by its weight, and add everything up. That single number is $r(t)$. Then slide one step forward in time and do it again.

A tiny worked case makes it concrete. Take $f = (0.5,\ 0.3,\ 0.2)$ and a stimulus that was $(\ldots,\ 4,\ 2,\ 6)$ over the last three steps, most recent last. Then

$$r \;=\; 0.5 \times 6 \;+\; 0.3 \times 2 \;+\; 0.2 \times 4 \;=\; 4.4,$$

the most recent value counting most. Slide along one step and repeat, and you have traced out $r(t)$.

Readers who have met signal processing will recognise this as a **convolution**. Note the **effect** of temporal filtering: fast variations in s get smoothed out in producing r.

Example I — the running average

Suppose the response is proportional to a running average over a window of n time steps. What does f look like?

Example II — the leaky average

Now suppose r depends on s averaged over time, but with a **memory that fades**. Points near t are weighted heavily; going further back, weights shrink.

The idea in plain words

"Leaky" is literal. Picture a bucket that inputs pour into while it drains through a hole in the bottom. What is in the bucket now reflects recent inflow strongly and old inflow weakly, because the old water has mostly leaked away. A membrane holds charge the same way, which is why this filter shape keeps reappearing.

13. Spatial Filtering

[Lecture slide omitted from the public copy.] Receptive field shapes. Retinal cells have concentric centre–surround fields; visual cortex cells have elongated, oriented ones. Both are nothing more than a weighting over space.

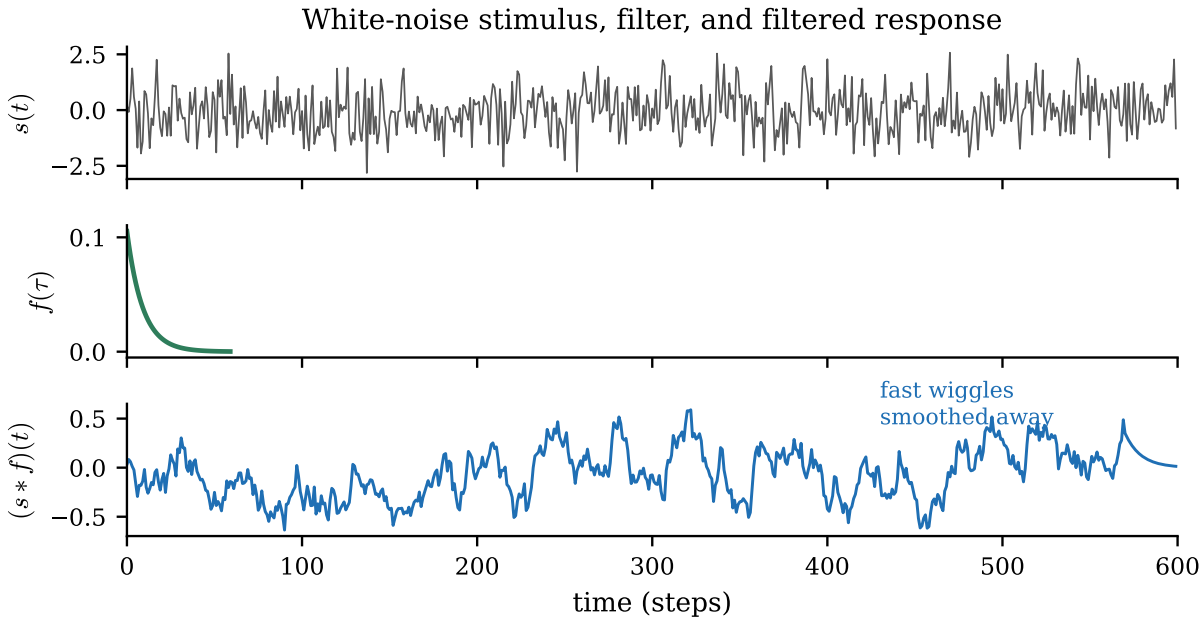


Figure 3: ***Filtering smooths.*** *A white-noise stimulus (top) convolved with a decaying filter (middle) gives the response (bottom). Each output point is a weighted sum over a window of recent input, so rapid fluctuations average out and only slower structure survives.*

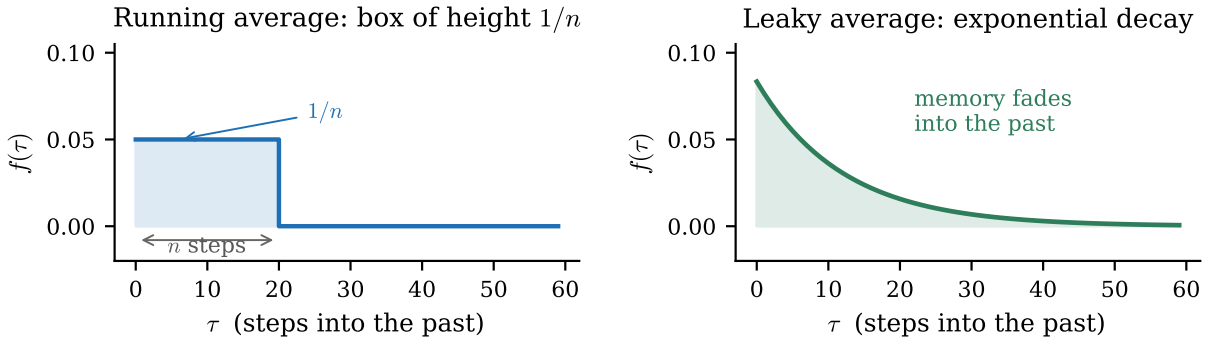


Figure 4: ***Two filter shapes.*** *The running average weights the last n steps equally at height $1/n$ and everything else at zero. The leaky average weights recent input heavily and older input less and less — a leaky integrator, which is how a cell membrane behaves.*

What if the neuron is sensitive to patterns of light *in space* rather than variations in time? The same idea applies — and you have already met it, as receptive fields.

Recall from Part I that retinal receptive fields have a **centre–surround** form, and that V1 neurons are sensitive to oriented edges. The goal now is to take temporal filtering and apply it to a stimulus s defined over space.

Consider a neuron whose receptive field is centred at (x, y). Neglecting time, take the stimulus — a pattern of light in two dimensions — weight it at each point by the receptive field f, and add everything up to get a *single number*, the predicted response:

Spatial filter

$$r(x,y) \;=\; \sum_{x'=-n}^{n} \sum_{y'=-n}^{n} s_{x-x',\,y-y'}\; f_{x',y'} \;=\; \iint dx'\,dy'\; s(x-x',\,y-y')\; f(x',\,y')$$

Again a convolution — now over spatial rather than temporal offsets.

The idea in plain words

Think of the receptive field as a small stencil of weights. Lay it over the image at position (x, y), multiply each pixel underneath by the weight sitting on it, add up the results, and write that one number down as the response at (x, y). Then slide the stencil to the next position and repeat. That sliding is all a convolution is.

So a ganglion cell's response is determined by **how similar the image is to its receptive field** f. "Similar" has an exact meaning here: it is precisely this weighting of the image by the filter.

The centre–surround filter, drawn properly

[Lecture slide omitted from the public copy.] A centre–surround filter as a surface: a positive peak in the middle with a shallow negative moat around it. Sketched beside it in the lecture is the same filter in cross-section, which is the easier way to see that the whole thing sums to roughly zero — and hence why it is blind to uniform illumination.

The cartoon draws f in 2D with shading standing for value. A better picture puts the *weight* into a third dimension: large **positive** values in the middle, **negative** values in the surround.

The interaction with an image is then easy to read off:

- a bright patch **at the centre** is multiplied by a large positive weight — this *increases* r;
- a bright patch **in the surround** is multiplied by a negative weight — this *decreases* r.

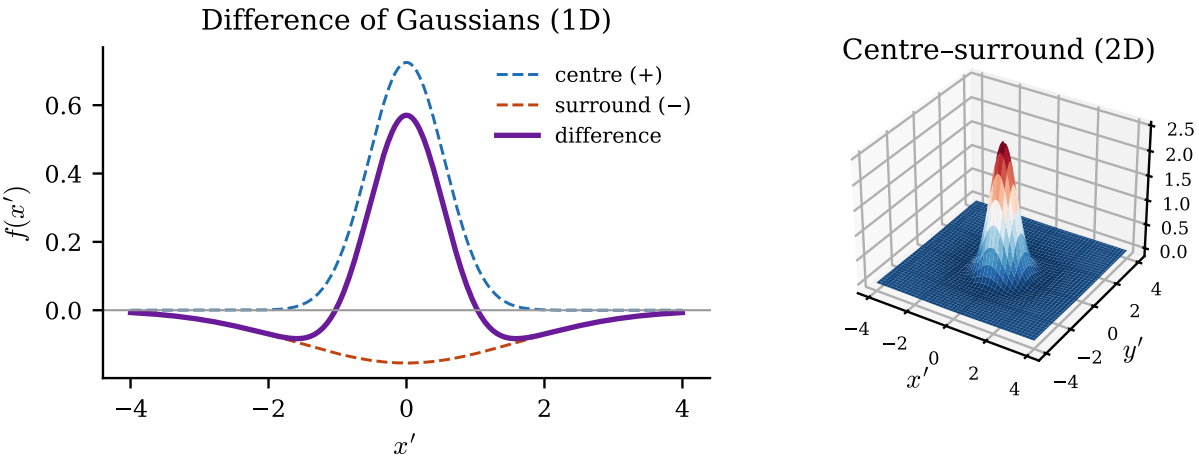


Figure 5: ***Centre–surround as a filter.*** *Left: a narrow positive Gaussian minus a broader, shallower one. Right: the same weights drawn as a surface over the two spatial coordinates — a positive peak in the middle, a negative moat around it.*

Difference of Gaussians

The centre–surround field is often approximated as a narrow positive Gaussian blob (the excitatory centre) **minus** a broader, shallower Gaussian (the suppressive surround).

What such a filter computes

[Lecture slide omitted from the public copy.] A difference-of-Gaussians filter applied to a photograph. Flat regions go blank and edges survive: the filter reports change, not absolute intensity.

The effect of a differencing filter is to **detect local changes**. It responds strongly when a bright patch lies near a dark patch, and it **cancels regions of constant intensity**.

This is not a peculiarity of biology. The same filters are standard in image processing — applying a difference-of-Gaussians filter in GIMP to a photograph returns an image consisting only of **edges**, for exactly this reason.

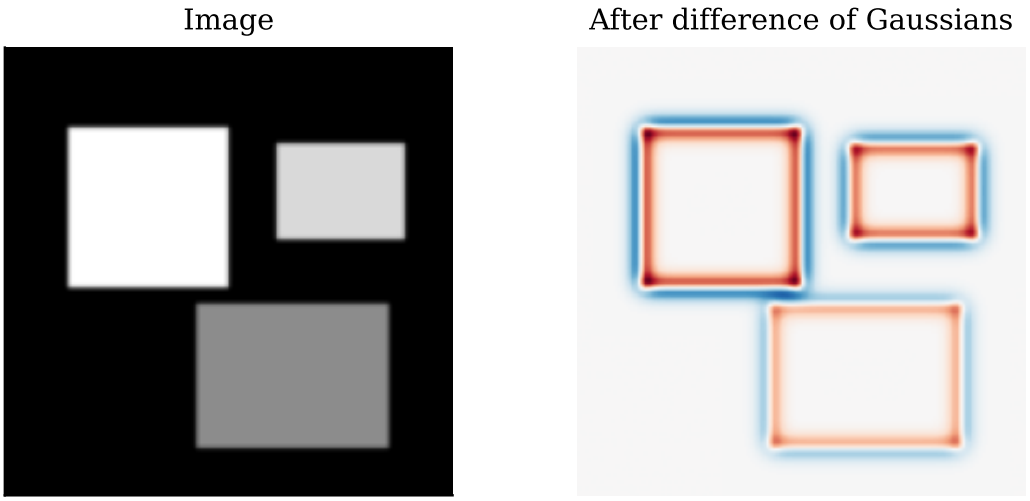


Figure 6: ***The filter detects change, not brightness.*** *Applying a difference of Gaussians to a synthetic image leaves only the boundaries. Inside a region of constant intensity the positive centre and negative surround cancel; at an edge they see different values and the cancellation fails.*

14. Spatiotemporal Filtering

[Lecture slide omitted from the public copy.] Spatiotemporal filtering. The receptive field becomes a stack of spatial filters, one per time lag, so the neuron is tuned to a pattern in space and time together.

In general a sensory neuron's response depends on *both* time and space. So the filter must depend on both:

$$r_{x,y}(t) \;=\; \iiint dx'\, dy'\, d\tau\; f(x', y', \tau)\; s(x - x',\, y - y',\, t - \tau)$$

The filter as a little movie

A spatiotemporal filter is a short 3D movie: a stack of frames, each showing the receptive field f at a different delay into the past.

15. Why a Linear Filter Cannot Be the Whole Story

Linear filtering is powerful, but it cannot be the full picture. Two questions expose the problem:

- **Can firing rates be negative?**
- **Can they increase indefinitely as the input increases?**

A linear operation permits both, and neither is true of a real neuron. A cell cannot fire fewer than zero spikes, and it cannot fire faster than its biophysics allows.

16. The Linear–Nonlinear Model

Both problems are fixed by one extra step that keeps linear filtering intact: a **nonlinear function applied to the filtered stimulus**.

Stage 1 — Linear filter *the what the neuron looks for*

Convolve the stimulus with f, collapsing the whole stimulus history into a single number:

$$(s * f)(t) \;=\; \int s(t-\tau)\, f(\tau)\, d\tau$$

Stage 2 — Static nonlinearity *the how it becomes a rate*

Pass that number through a fixed **input/output function** g:

$$r(t) \;=\; g\!\left(\int s(t-\tau)\, f(\tau)\, d\tau\right)$$

With a sigmoid g: when $s * f$ is small or negative the rate goes to **zero**; when it is very large the rate **saturates**.

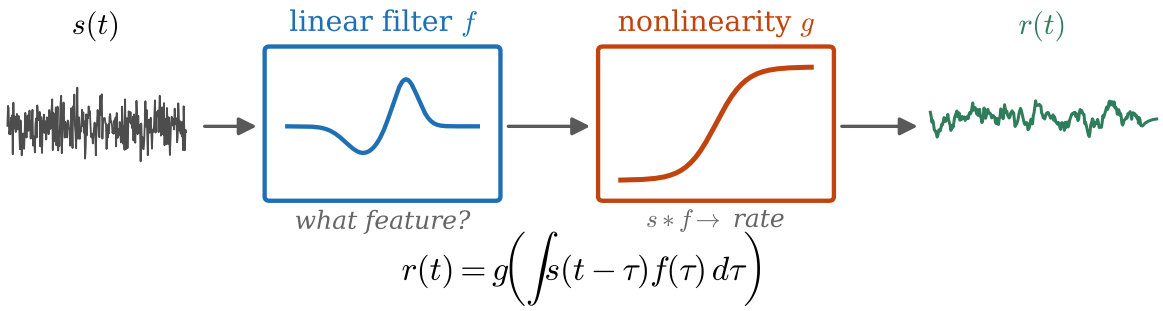


Figure 7: ***The linear–nonlinear model.*** *The stimulus is projected onto a filter f, collapsing its whole recent history into one number; that number passes through a fixed input/output function g, which clamps the output at zero below and saturates it above.*

g is called **static** because it acts instantaneously on the current filter output: all the memory in the model lives in f.

Closing Part II. We have a two-component model: a **linear filter** f saying *what feature* the neuron looks for, and a **static nonlinearity** g turning the filtered signal into a rate. But we have only said what the components *are*, not how to obtain them from data. That is Part III.

Part III

Finding the Model from Data

Stimuli as vectors, why white noise, the spike-triggered average, recovering the nonlinearity by Bayes' rule, PCA for multiple features, and maximally informative dimensions.

17. The Problem Is Dimensionality

We want $P(\text{response} \mid \text{stimulus})$, sampling the system's responses to many stimuli so we can characterise what about the input triggers responses. The obstacle is that a stimulus is an enormous object.

A stimulus is a vector

Discretise a stimulus waveform in time: sample it at n points to get $s = (s_{t_1}, s_{t_2}, \ldots, s_{t_n})$. That list of n numbers *is* a point in an n-dimensional space.

The idea in plain words

Two numbers specify a point on a plane; three specify a point in a room. A stimulus chopped into 100 time points is a point in a 100-dimensional space — the first axis is "how big was the stimulus at time 1," the second "how big at time 2," and so on. Each whole stimulus is **one dot** in that space.

Watch out — axis labels versus axis units

We often label the axes by their times $(t_1, t_2, \ldots)$, which makes it tempting to think of the axes as time. They are not. Each axis carries the units of the *y-axis* of the original waveform — an intensity, a voltage. Time only indexes *which* axis.

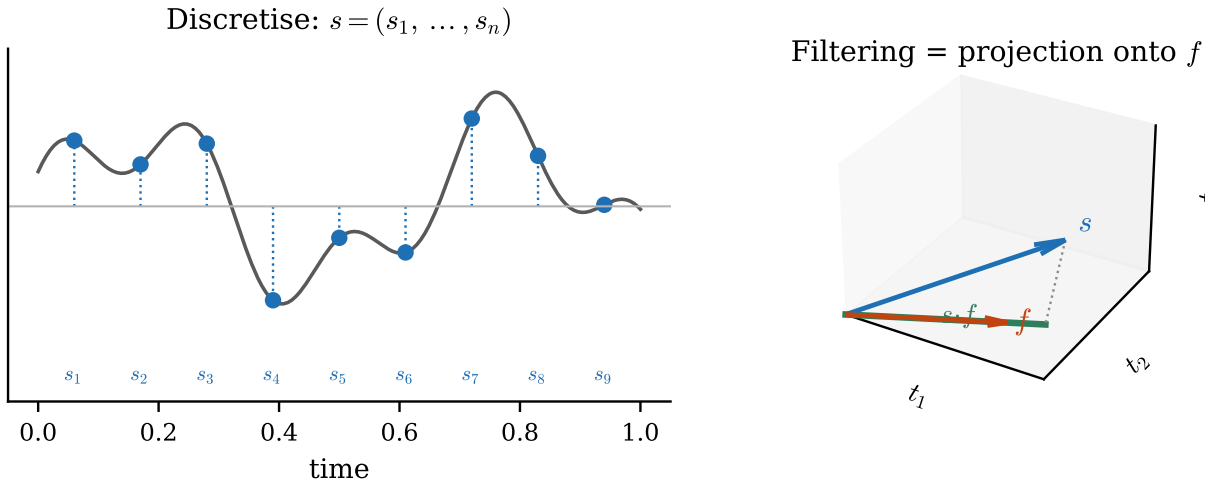


Figure 8: ***A stimulus is a point in a high-dimensional space.*** *Sampling a waveform at n times gives n numbers, and those are coordinates: one axis per time point. The filter f lives in the same space, and filtering projects the stimulus onto it — turning n dimensions into one.*

Filtering is projection

This viewpoint pays off immediately. The filter f is also a list of n numbers, so it too is a vector in the same space. And the filtering operation $\sum_k s_{t-k} f_k$ is a sum of products — a **dot product**.

Linear filtering = convolution = projection

Applying a filter asks: *how much of this stimulus lies along the direction f?* It takes an n-dimensional stimulus and returns one number — its **projection** onto f. This is exactly the dimensionality reduction we need: from n dimensions down to one.

18. Why White Noise?

To sample the system's responses we need a stimulus that explores the space broadly. The standard choice is **white noise**, and the reason is worth understanding rather than memorising.

The Gaussian, briefly

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-x_0)^2}{2\sigma^2}\right)$$

Here x_0 is the **centre** of the function and σ a measure of its **width**. Reading $p(x)$ as a probability distribution, the mean is $\bar{x} = x_0$, the variance $\langle (x-\bar{x})^2 \rangle = \sigma^2$, and the standard deviation is σ.

The key property

If you add together two or more Gaussian random numbers, the result is *also* Gaussian. And adding together weighted stimulus values at different times is exactly what filtering does.

The idea in plain words

So white noise is Gaussian *no matter how you filter it.* Every direction in the stimulus space looks statistically the same — the cloud of stimuli is a symmetric ball with no special directions built in. That matters because we are about to hunt for special directions. If the stimulus itself had preferred directions, we could not tell whether a direction we found belonged to the neuron or to our own choice of stimulus.

Watch out — white noise is a convenience, not a fact about brains

Real neurons do not live in a world of white noise, and **the statistics of the stimulus you use do affect the model you arrive at**. We use white noise because it is unbiased, not because it is natural. Maximally informative dimensions, below, is the method that escapes this restriction.

19. The Spike-Triggered Average

[Lecture slide omitted from the public copy.] Reverse correlation. Collect the stimulus segment preceding each spike — the spike-conditional ensemble — and average them. From Dayan and Abbott.

Drive the neuron with white noise. Every time it spikes, grab the stretch of stimulus immediately preceding that spike. The collection of all such stretches is the **spike-conditional ensemble**. Average them:

Spike-triggered average (STA)

$$\mathrm{STA}(\tau) \;=\; \frac{1}{N}\sum_{i=1}^{N} s(t_i - \tau)$$

the average stimulus at a time τ *before* a spike, over all N spikes. Also called **reverse correlation**.

The idea in plain words

Stimulus features unrelated to spiking point in random directions, so averaging many of them cancels them toward zero. Whatever survives the averaging is what was reliably present before spikes — the neuron's preferred feature. In the vector picture: the cloud of spike-triggering stimuli sits off-centre from the full stimulus cloud, and the STA is the **centre of mass** of that displaced cloud.

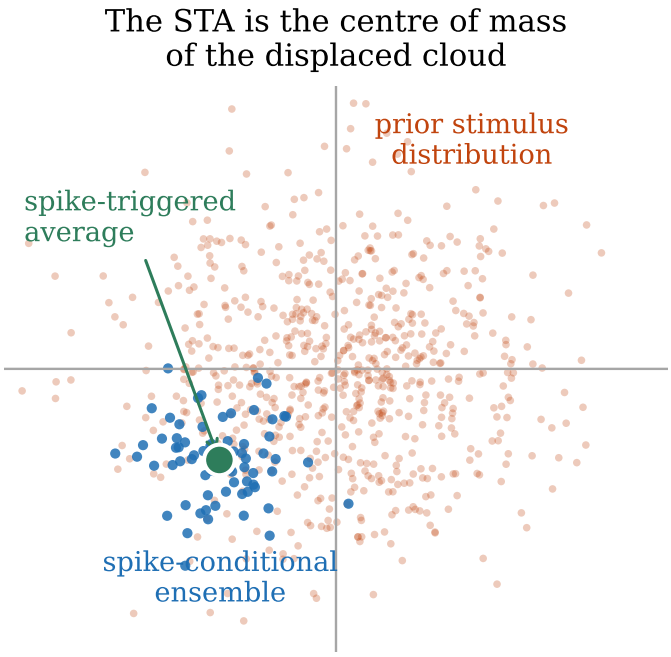


Figure 9: ***The STA, geometrically.*** *Red: all stimuli (the prior). Blue: only those that preceded a spike. Their centre of mass (green) is displaced from the origin, and that displacement is the spike-triggered average — the neuron's preferred feature.*

20. Recovering the Nonlinearity

[Lecture slide omitted from the public copy.] Recovering the input/output function with Bayes' rule. Left: when the spike-conditional distribution sits on top of the prior, the feature is uninformative and the resulting curve is flat. Right: when the two separate, their ratio gives a rising nonlinearity.

The STA gives us the filter f. Now for g. Project every stimulus onto f to get a single number s_1, and ask for $P(\text{spike} \mid s_1)$ — which is precisely the input/output function.

This comes straight from **Bayes' rule**:

$$P(\text{spike} \mid s_1) \;=\; \frac{P(s_1 \mid \text{spike})\; P(\text{spike})}{P(s_1)}$$

The idea in plain words

All three quantities on the right are things you can just *count* from the data. $P(s_1)$ is the histogram of the projection over *all* stimuli — the **prior**. $P(s_1 \mid \text{spike})$ is the histogram over only the stimuli that produced a spike — the **spike-conditional** distribution. $P(\text{spike})$ is just the overall fraction of time bins containing a spike. So the nonlinearity is, up to a constant, the **ratio of two histograms**.

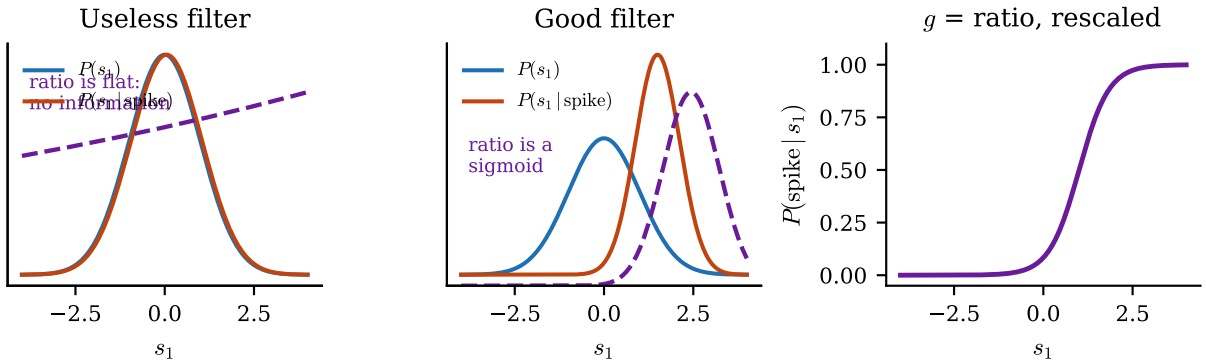


Figure 10: ***The nonlinearity is a ratio of two histograms.*** *Left: prior and spike-conditional distributions nearly coincide, so their ratio (dashed) is flat — the filter tells you nothing. Middle: they separate, and the ratio acquires a threshold. Right: that ratio, rescaled, is g.*

When have we found a good filter?

This ratio also tells us whether our filter is any good:

- If the prior and the spike-conditional distribution are **very similar**, their ratio is a *flat line*. Knowing the projection tells you nothing about whether a spike occurred — a useless filter.
- If they are **very different**, the ratio has interesting structure — a threshold, a sigmoid. This is a filter worth having.

The idea in plain words

This is the criterion behind everything that follows. A good feature is one where *knowing it changes your belief about spiking.* Separation between the two distributions is exactly that change, made visible.

21. Multiple Features: Principal Component Analysis

[Lecture slide omitted from the public copy.] PCA on faces. The leading principal components of a set of face images — eigenfaces — are the directions along which faces differ most from one another.

[Lecture slide omitted from the public copy.] PCA used for spike sorting (Koepsell et al. 2009). Projected onto the first two principal components, waveforms from different neurons fall into separate clusters. The shapes drawn at the right are the kind of distinct spike waveforms those clusters correspond to.

The STA gives a simple view of the stimuli leading up to a spike, but its very simplicity is a limitation.

Watch out — the STA can be zero for a perfectly well-tuned cell

If a cell responds to *both* positive-to-negative changes **and** negative-to-positive changes, the spike-triggering stimuli come in opposite-signed pairs. Averaging them gives approximately **zero** — a flat line — even though the neuron is strongly and reliably tuned. The mean is simply the wrong statistic here.

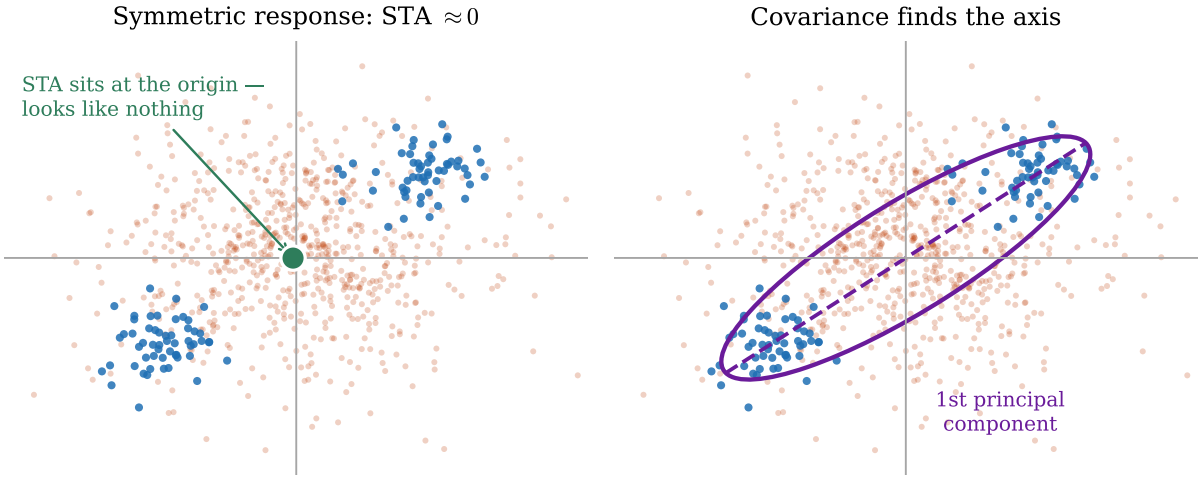


Figure 11: ***Where the STA fails.*** *If a cell responds to a feature* and *its negative, the spike-triggering stimuli form two opposite lobes. Their mean sits at the origin (left), so the STA looks like nothing. The covariance still finds the axis they lie along (right).*

The fix is to look at the *spread* of the spike-conditional cloud rather than its centre — its **covariance**. If the cloud is stretched along certain directions relative to the prior, those directions are features the neuron cares about.

Principal component analysis (PCA)

Finds the directions along which the variation of the data is maximal, and thereby gives a lower-dimensional representation — a computationally easier problem. Geometrically: fit an ellipsoid to the cloud and read off its axes, longest first.

With n features the model generalises to

$$r(t) \;=\; g\big(f_1 * s,\; f_2 * s,\; \ldots,\; f_n * s\big),$$

where g is now a function of several variables — a surface rather than a curve.

The same technique appears all over the place: **eigenfaces** (principal components of a set of face images) and **spike sorting** (clustering recorded waveforms by their first two principal components to decide which spikes came from which neuron).

22. Maximally Informative Dimensions

PCA still leans on the stimulus being Gaussian. Can we go directly at the thing we actually care about — the *difference* between the prior and the spike-conditional distribution — and search for the filter that maximises it?

Kullback–Leibler divergence

The standard measure of the difference between two probability distributions:

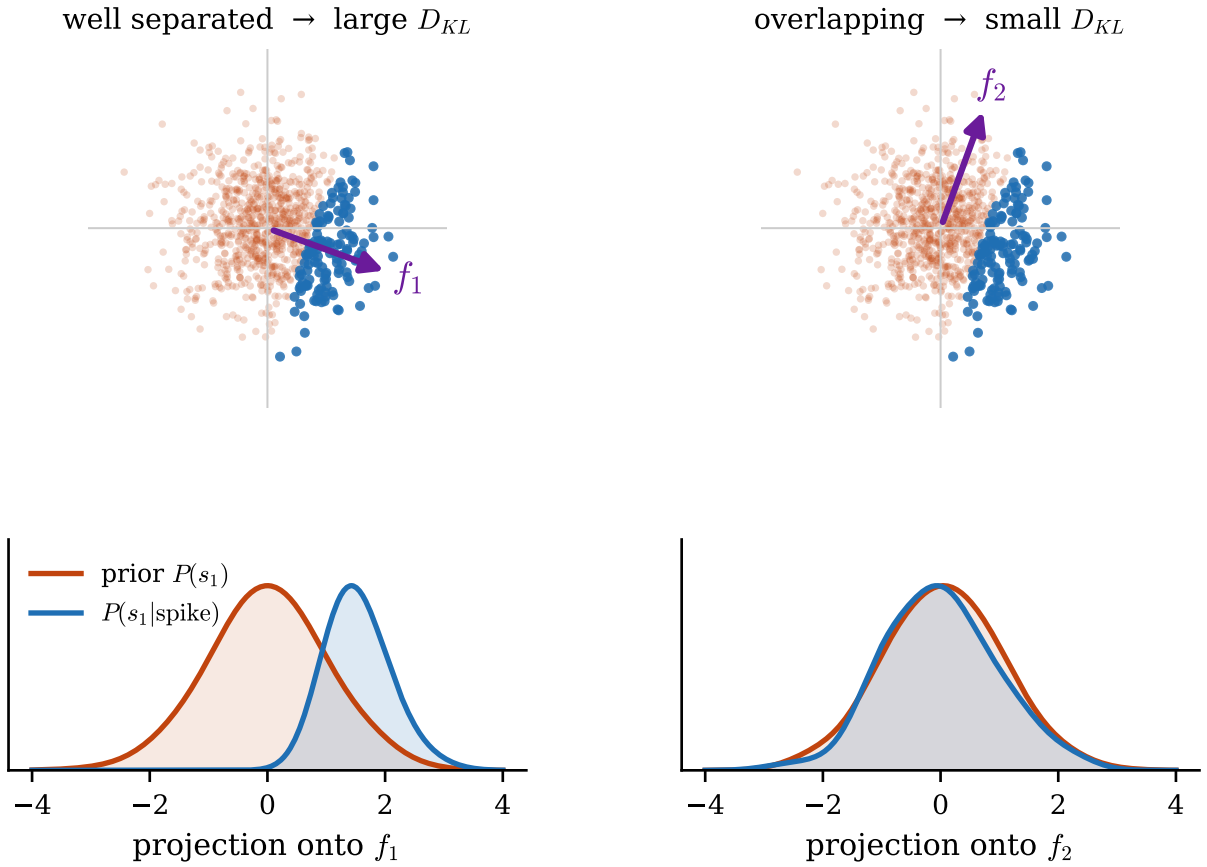
$$D_{\mathrm{KL}}(P \,\|\, Q) \;=\; \int ds\; P(s)\, \log \frac{P(s)}{Q(s)}$$

The method, developed by **Tatyana Sharpee and Bill Bialek**, works like this. Take a candidate filter f, a vector in the high-dimensional stimulus space. Project all stimuli onto it to get the prior, and project the spike-triggering stimuli onto it to get the spike-conditional distribution. Compute D_{KL} between them. Then rotate f and try again — searching for the direction that makes the two distributions as different as possible.

The idea in plain words

Instead of computing a filter with a formula (average them; take the biggest variance), you *search* for one. Point in a direction, look at how well spikes are separated from non-spikes along it, and keep turning until the separation is as large as it can get.

This turns out to be equivalent to **maximising the mutual information between the spike and the stimulus**: we are looking for the stimulus component that is as informative as possible, so that observing a spike pins down the stimulus as tightly as possible.



Rotate f and keep the direction that drives the two distributions furthest apart.

Figure 12: ***Maximally informative dimensions.*** *For each candidate filter, project all stimuli (red) and the spike-triggering ones (blue) onto it and measure how different the two distributions are. f_1 separates them; f_2 barely does. The search keeps the direction maximising D_{KL}.*

Watch out — the cost of the method

The maximisation step is **not guaranteed to converge to a unique maximum**. It is a hard optimisation problem, and you may land in a local optimum. Power over generality, paid for in reliability.

Three ways to find a filter. **STA** — one filter, from the mean of the spike-triggered ensemble. **PCA** — multiple filters, from its covariance. **MID** — an information-theoretic search using the whole distribution, which removes the requirement for Gaussian stimuli.

Part IV

From Firing Rates to Spikes

Lecture 3 — the coin-flip model, binomial to Poisson, the Fano factor, where variability comes from, and generalized linear models.

23. The Assumption We Have Been Hiding

Everything so far modelled a *time-varying firing rate* $r(t)$. But the data are **spike times**. A smooth function $r(t)$ can miss real fine structure in a spike train.

So: how do we get from $r(t)$ to actual spikes? The assumption is simple:

The spike-generation assumption

Every spike is generated **independently**, with a probability scaled by the time-varying rate $r(t)$.

What does this mean, and how can we test it? Start from the most elementary random process there is.

24. From Coin Flips to the Poisson Distribution

Step 1 — a biased coin

A fair coin lands heads with probability $1/2$. Take instead a biased coin with a small probability p of landing heads — and say the system "spikes" when it comes up heads.

Step 2 — many bins, the binomial

Divide a total time T into n small bins of size Δt, so $n = T/\Delta t$. Assume the same probability p of firing in each bin. How many spikes occur in total?

That number is random and differs every trial. It follows the **binomial distribution** — "binomial" because each bin has two outcomes, firing with probability p and not firing with probability $1 - p$:

Binomial distribution

$$P(k \text{ spikes in } n \text{ bins}) \;=\; \binom{n}{k} p^k \,(1-p)^{\,n-k}, \qquad \binom{n}{k} = \frac{n!}{k!\,(n-k)!}$$

The idea in plain words

Build it piece by piece. You need a spike in exactly k bins: probability p^k. You need *no* spike in the remaining $n-k$ bins: probability $(1-p)^{n-k}$. And you do not care *which* k bins they were, so count the number of ways to choose k bins out of n — that is $\binom{n}{k}$. Multiply the three together.

(Recall $3! = 3 \times 2 \times 1$, and in general $n! = n(n-1)(n-2)\cdots 1$.) For the binomial, the mean number of spikes is np — number of bins times probability per bin — and the variance is $np(1-p)$.

Step 3 — the limit, and Poisson

[Lecture slide omitted from the public copy.] The Poisson distribution at three mean rates. A low rate gives a skewed distribution peaked near zero; higher rates become broader and roughly symmetric.

Now let the bins get very small and the per-bin probability very small with them. To keep something finite, define a **rate**

$$r \;=\; \frac{p}{\Delta t}$$

— probability per unit time — and hold it fixed as $\Delta t \to 0$. In that limit the binomial becomes the **Poisson distribution**:

Poisson distribution

$$P_T(k) \;=\; \frac{(rT)^k\, e^{-rT}}{k!}$$

now indexed by the total time T rather than the number of bins.

Its properties

- **Mean** $= rT$. Intuitive: number of spikes is rate times time.
- **Variance** $= rT$. Less intuitive — and note it is *the same as the mean.* That is a very unusual property.

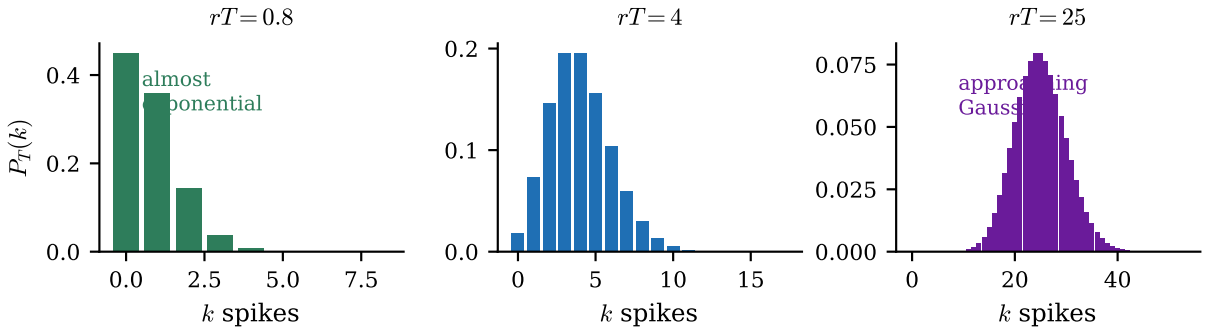
Fano factor

$$F \;=\; \frac{\text{variance}}{\text{mean}}$$

$F = 1$ for a Poisson process. This makes F the standard test of whether a spike train is Poisson.

The interval distribution

If spikes really are generated independently in each bin with probability $r\,\Delta t$, then the intervals between successive spikes follow an **exponential distribution**.



Mean $= rT$ and variance $= rT$ — equal, which is what makes $F = \sigma^2/\mu = 1$ a test for Poisson.

Figure 13: ***The Poisson distribution at three rates.*** *At a low rate it is almost exponential, with zero spikes the most likely outcome; as the rate rises it becomes increasingly Gaussian-looking.*

The idea in plain words

You can see where this comes from by reading the Poisson formula for the *first* spike as a function of time: the exponential factor e^{-rT} survives and the factorial disappears. The interval distribution lacks the leading power of T because it must normalise over all time.

So there are **two strong signatures** of a Poisson process: a Fano factor of 1, and an exponential interval distribution.

As an aside on shape: for a *low* firing rate the Poisson distribution looks almost exponential, while as the rate rises it looks more and more Gaussian.

25. Testing It Against Real Data

[Lecture slide omitted from the public copy.] Testing Poisson against area MT (Shadlen and Newsome, 1998). Raster and PSTH, the interspike interval distribution, and variance against mean spike count — the last lying close to, but a little above, the Poisson line.

In general the rate varies over time, so any test must allow r to vary.

Take data from a neuron in monkey **MT cortex**, which is sensitive to motion, while the monkey watches drifting patterns. The same pattern is shown over and over. As in the retina there is an overall modulation of firing rate over time — but zoom in on a short interval and the spikes are very variable.

Now split the data into short windows and plot, for each window, the mean spike count against the variance. **The slope of that plot is the Fano factor.** If the spikes were Poisson — each window with its own rate — the slope should be about 1. And in the data it is: the points sit very close to the line of slope one.

The idea in plain words

So even though the firing rate is changing from moment to moment, within each short chunk the cell's response looks Poisson. The rate carries the signal; the spikes around it are as random as coin flips.

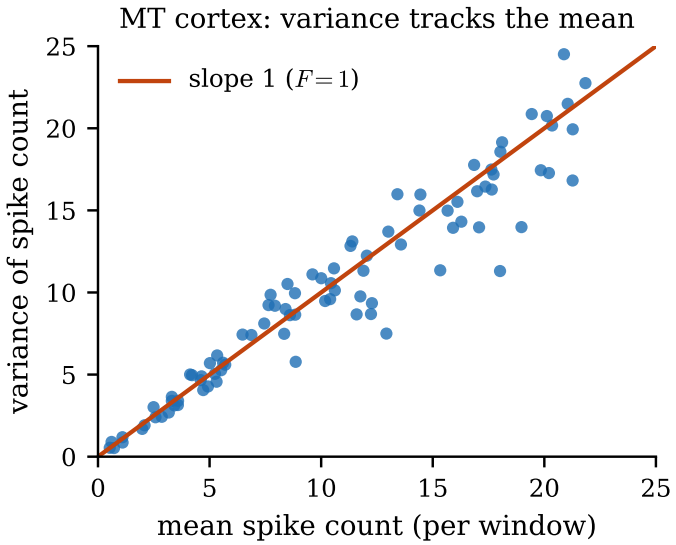


Figure 14: ***The Fano factor in data.*** *Splitting an MT recording into short windows and plotting each window's spike-count variance against its mean: the points follow the line of slope 1. Even as the rate changes over time, within each chunk the response looks Poisson.*

Where does the variability come from?

Likely explanation: while the neuron receives a mean input proportional to the stimulus, it is also receiving a **barrage of background input**. A cortical neuron gets inputs from around **10,000** other neurons. If that input is **balanced** — varying around zero, both positive and negative — it will not add much to the average firing rate, but it will *jitter* the spikes.

A model neuron driven by white noise behaves the same way: its interspike interval distribution looks close to exponential. Plotting the count of intervals on a **log** axis against interval makes this easy to check — an exponential becomes a **straight line** with negative slope given by the firing rate.

The idea in plain words

So the Poisson randomness we build into response models is not laziness. It is standing in for the effect of a large amount of real but unobserved background input.

Where it breaks: short intervals

Zoom in on very short intervals and the distribution **stops** looking exponential. The reason is good: a neuron cannot fire arbitrarily rapidly. Biophysical processes prevent it from firing immediately after an action potential, leaving a visible gap between successive spikes.

26. Generalized Linear Models

[Lecture slide omitted from the public copy.] The generalized linear model (Pillow et al., Nature 2008). Stimulus filter, exponential nonlinearity, stochastic spiking — plus a post-spike filter feeding the cell's own recent spikes back into its input.

If intrinsic limitations matter, take them seriously — these processes can add a lot of structure to spike trains. A neuron might have a **resonance**, preferring to fire at a certain frequency independently of the stimulus.

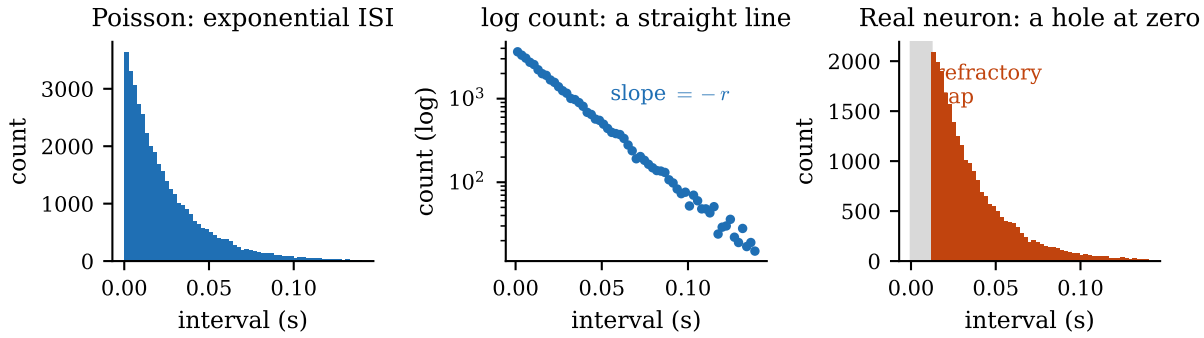


Figure 15: ***Interspike intervals.*** *A Poisson process gives exponentially distributed intervals (left), which appear as a straight line on a log count axis (middle). A real neuron departs from this at short intervals: the refractory period carves a hole near zero (right).*

Generalized linear model (GLM)

The familiar chain — stimulus, filter, nonlinearity — plus two additions: an explicit **Poisson spike-generation step**, and a **post-spike filter** that feeds each generated spike back into the input of the nonlinearity.

$$r(t) \;\propto\; \exp\big(f * s \;+\; h * (\text{spike history})\big)$$

The idea in plain words

The post-spike filter is the neuron talking to its own future. Suppose you want refractoriness: make the filter a big **negative** pulse that decays back over time. Right after a spike it drags the cell hard away from threshold, then releases it. The one used in the literature is more sophisticated — it dips negative first (refractoriness), then goes **positive**, promoting a spike some time later. That gives a neuron a slight tendency to fire periodically.

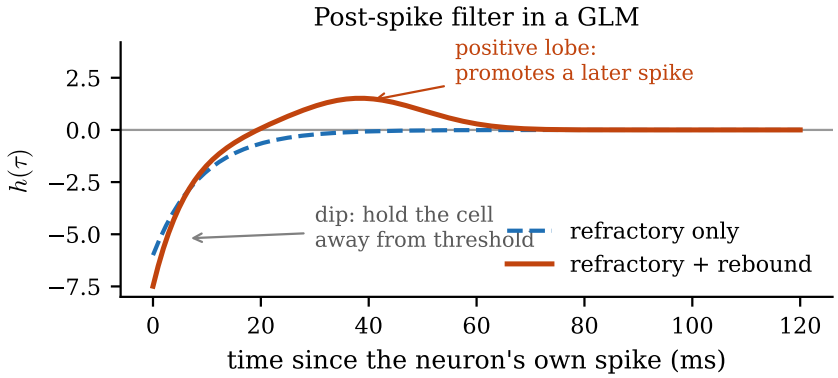


Figure 16: ***The post-spike filter.*** *After each spike this waveform is fed back into the neuron's own input. A pure dip implements refractoriness. The filter used in practice dips, then rises above zero, promoting a spike a little later — giving the cell a tendency to fire rhythmically.*

Why an exponential nonlinearity?

In earlier models g was computed directly from the data. In the GLM it is *fixed* as an exponential. The reason is a result of **Liam Paninski**'s: fixing the nonlinearity to be exponential (or in the exponential family) makes finding all the model's parameters an optimisation problem that is

globally convergent.

Watch out — a real trade being made

You sacrifice generality — you no longer read g off the data — in exchange for a model that is more complete in another way. You can add more filters, and the fit is guaranteed to be solved reliably and repeatably.

Adding more influences

[Lecture slide omitted from the public copy.] Coupling filters let one neuron's spikes influence another's, so the model can capture correlations that no single-cell model reaches. Pillow et al., Nature 2008.

Why stop at self-history? As **Emery Brown** and colleagues pointed out, one can include many other intrinsic and extrinsic factors — refractory effects *and* the firing of other neurons in the network. Applied to retinal data, including both the neuron's own output and its neighbours' let them predict detailed spike-interval patterns *and* the correlations between neurons, with great accuracy.

27. The Time-Rescaling Theorem

[Lecture slide omitted from the public copy.] The time-rescaling theorem (Brown et al. 2001). If the model's rate is right, rescaling each interspike interval by that rate should give an exponential distribution — a straight line on log axes.

A last idea from Emery Brown's group, and a genuinely elegant one. We can use the Poisson nature of firing to test whether our model has captured everything.

Take a fitted model — a GLM whose output depends on the stimulus, the neuron's own firing history, and other neurons' firing. Take the observed intervals between successive spikes, and **scale each one by the firing rate the model predicts** at that moment.

The idea in plain words

If the model truly accounts for every influence on firing, then all the structure has been explained away, and what is left over should be pure randomness. Concretely: the rescaled intervals should look like a clean exponential — a pure Poisson process of rate 1. Any remaining structure is something the model has missed.

Time-rescaling theorem

Rescaling observed interspike intervals by the model's predicted rate should yield a pure Poisson process with effective rate 1. Deviation from a clean exponential measures what the model still fails to capture.

28. What These Models Leave Out

Worth recording, because it is the lecture's own closing caution.

These methods are a powerful way of thinking about the neural code, but there is a lot they ignore. In particular they give the impression that **a neuron represents one particular thing, and that is it**. In fact neural responses are modulated by many other influences:

- how the animal is using its body to deploy its senses;
- what it *expects* to see in the environment;
- the context in which the stimulus appears.

These approaches have made huge progress in making sense of a great deal of data, even if under rather limited circumstances. But it is entirely possible that some of these ideas will be overturned by a more general approach. The field remains wide open.